

Power Profile

Building Energy Analysis Platform — Full Technical Report

Graduation Project | Full-Stack Web Application | Backend: Laravel 13 (PHP 8.3) | Frontend: React 18 + Vite | Database: SQLite | Report Date: May 2026

1. Project Overview

Power Profile is a full-stack web application designed to help engineers and architects model, analyse, and manage the electrical power consumption of buildings. It provides a structured hierarchy (Project → Building → Floor → Room) where electrical components are assigned at every level, and the system computes load profiles, peak demand, and optimal dispatch of power sources (solar, grid, generator).

Core Capabilities

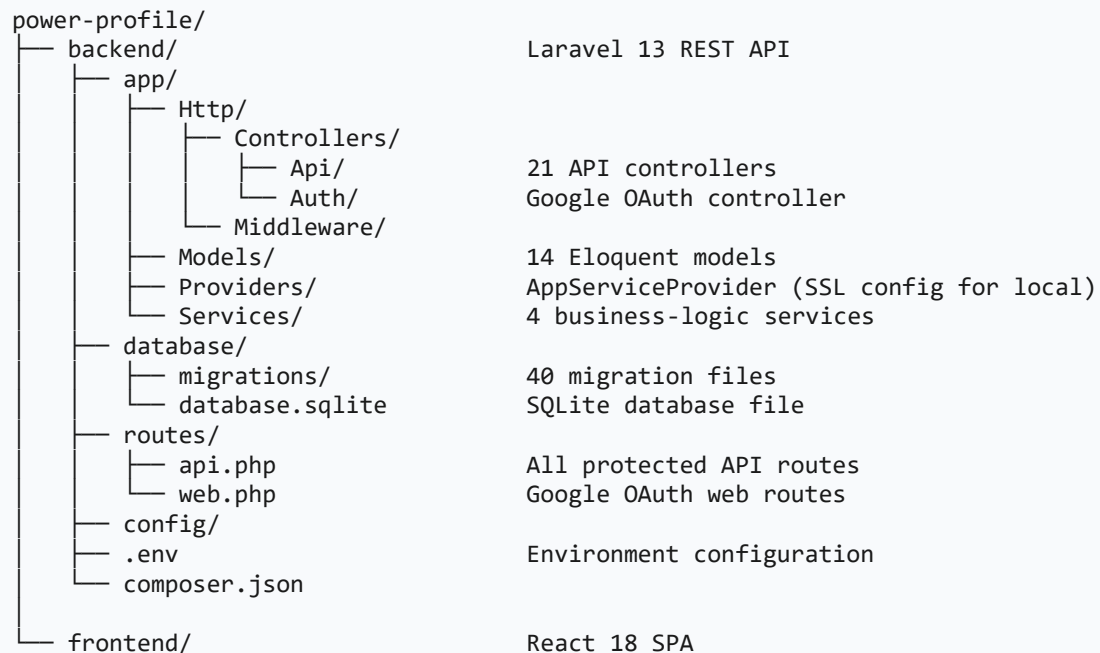
- Define multi-building projects with a four-level hierarchy
- Attach electrical components (AC units, lighting, motors, etc.) at any level
- Model power sources: utility grid lines, generator backup lines, solar panels
- Calculate 24-hour load profiles based on component usage schedules
- Simulate priority-based dispatch (Solar → Grid → Generator)
- Backup and restore configurations (local file or server storage)
- Collaborate with a team using role-based access control
- Google OAuth authentication — no separate password management needed

Technology Stack

Layer	Technology	Version
Backend Framework	Laravel	13.x
Backend Language	PHP	8.3+
API Authentication	Laravel Sanctum	4.3
OAuth	Laravel Socialite (Google)	5.26

Database	SQLite	3.x
Frontend Framework	React	18.2
Routing (Frontend)	React Router DOM	6.30
Build Tool	Vite	4.4
CSS Framework	Tailwind CSS	3.4
HTTP Client	Axios	1.13
Charts	Recharts	3.8

2. Project Folder Structure



src/	
pages/	15 page components
components/	13 reusable components
api/	
axios.js	Axios instance + token injection
contexts/	
AuthContext.jsx	Global auth state
utils/	
navContext.js	In-memory navigation state
downloadJson.js	JSON file download helper
App.jsx	Route definitions
main.jsx	Entry point
vite.config.js	
tailwind.config.js	
package.json	

3. Database Schema

The database uses SQLite with 15 tables. The design follows a strict hierarchy for buildings, a polymorphic pattern for shared power-source entities, and a separate component-type catalogue for reusable presets.

3.1 Entity Hierarchy Tables

Table	Key Fields	Purpose
<code>users</code>	id, name, email, password (nullable), google_id, avatar, google_token, google_refresh_token, email_verified_at, is_admin	User accounts. Accounts are created automatically on first Google login.
<code>projects</code>	id, user_id, name, building_type, current_step, buildings_count, total_power, solar_power, existing_solar_power, solar_source, generator_power, generator_source, auto_backup_interval, last_auto_backup_at, work_days (JSON), work_time_intervals (JSON), working_season_intervals (JSON), location_lat, location_lng, location_name	Top-level project. Owns buildings. Holds project-wide schedule and location for solar calculations.
<code>buildings</code>	id, project_id, name, floors (count), area, power_consumption, solar_power, existing_solar_power, solar_source, generator_power, work_days, work_time_intervals, working_season_intervals	Building inside a project. Inherits or overrides schedule from project.

<code>floors</code>	id, building_id, name, area, power_consumption, solar_power, existing_solar_power, solar_source, generator_power, work_days, work_time_intervals, working_season_intervals	Floor inside a building.
<code>rooms</code>	id, floor_id, name, area, power_consumption, solar_power, existing_solar_power, solar_source, generator_power, work_days, work_time_intervals, working_season_intervals	Room inside a floor. Lowest level in the hierarchy.

3.2 Component Tables

Components exist at four levels. Each table mirrors the same set of fields but with a different parent FK. A shared `component_types` catalogue provides preset defaults.

Table	Parent FK	Shared Component Fields
<code>project_components</code>	project_id	component_type_id, power (W), phases (1phase/3phase), power_factor (0–1), quantity, group_name, priority (critical/normal/non_critical), needs_socket (bool), usage_season (all/summer/winter), usage_day_type (all/weekday/weekend), usage_time_intervals (JSON array of {start, end})
<code>building_components</code>	building_id	
<code>floor_components</code>	floor_id	
<code>room_components</code>	room_id	
Table	Key Fields	Purpose
<code>component_types</code>	id, name (unique), is_preset, default_power, default_phases, default_power_factor, default_needs_socket, default_usage_season, default_usage_day_type, default_usage_time_intervals	Master catalogue of component definitions. Preset types ship with the app; custom types are created per-user.

3.3 Power Source Tables (Polymorphic)

Each of the three power-source tables uses Laravel's polymorphic relation, meaning a single table holds rows for project, building, floor, and room owners.

Table	Polymorphic Columns	Other Fields	Purpose
utility_lines	lineable_type, lineable_id	name, power (VA capacity), phases	Utility grid connections (mains supply).
generator_lines	generable_type, generable_id	name, power, phases	Diesel / backup generator capacity.
sockets	socketable_type, socketable_id	phase_type, power (VA per outlet), quantity	Electrical outlets; demand computed with diversity factors.

3.4 Support Tables

Table	Key Fields	Purpose
project_users	project_id, user_id, role (admin/main/normal)	Project membership and permissions. Project creator is always admin.
server_backups	project_id, entity_type, entity_id, entity_name, created_by, data (longText JSON)	Stores full JSON snapshots of any entity on the server for later restore.
personal_access_tokens	tokenable_type, tokenable_id, name, token (hashed), abilities, last_used_at, expires_at	Laravel Sanctum API bearer tokens (one token per user session).

4. Backend API Reference

All routes below (except admin login and Google OAuth) are protected by `auth:sanctum` middleware. Clients must include the header `Authorization: Bearer {token}`.

Authentication (Web Routes)

Method	URL	Description
GET	/auth/google	Redirect user to Google consent screen
GET	/auth/google/callback	Handle Google callback, create/update user, return Sanctum token to frontend
GET	/api/user	Get authenticated user profile
POST	/api/logout	Revoke current Sanctum token

Projects

Method	URL	Description
GET	/api/projects	List all projects owned by or shared with the user
POST	/api/projects	Create a new project
GET	/api/projects/{project}	Get project details
PUT	/api/projects/{project}	Update project (name, schedule, location, solar/generator settings)
DELETE	/api/projects/{project}	Delete project (admin only)
GET	/api/projects/{project}/all-floors	List every floor in the project (all buildings)

GET	/api/projects/{project}/all-rooms	List every room in the project (all floors/buildings)
-----	-----------------------------------	---

Buildings

Method	URL	Description
GET	/api/projects/{project}/buildings	List buildings in project
POST	/api/projects/{project}/buildings	Create building
PUT	/api/projects/{project}/buildings/{building}	Update building
DELETE	/api/projects/{project}/buildings/{building}	Delete building

Floors

Method	URL	Description
GET	/api/buildings/{building}/floors	List floors in building (includes building + project metadata)
POST	/api/buildings/{building}/floors	Create floor
PUT	/api/buildings/{building}/floors/{floor}	Update floor
DELETE	/api/buildings/{building}/floors/{floor}	Delete floor

Rooms

Method	URL	Description
GET	/api/floors/{floor}/rooms	List rooms in floor (includes floor + building + project metadata)
POST	/api/floors/{floor}/rooms	Create room
PUT	/api/floors/{floor}/rooms/{room}	Update room
DELETE	/api/floors/{floor}/rooms/{room}	Delete room

Components (same pattern at 4 levels)

Method	URL Pattern	Description
GET	/api/{entity}/{id}/components	List components. Entity = projects, buildings, floors, rooms
POST	/api/{entity}/{id}/components	Add component
PUT	/api/{entity}/{id}/components/{component}	Update component
DELETE	/api/{entity}/{id}/components/{component}	Remove component
GET	/api/component-types	List all preset and custom component types

Power Sources (same pattern for utility-lines, generator-lines, sockets)

Method	URL Pattern	Description
--------	-------------	-------------

GET/POST	/api/{entity}/{id}/utility-lines	List or add utility grid connections
GET/POST	/api/{entity}/{id}/generator-lines	List or add generator backup lines
GET/POST	/api/{entity}/{id}/sockets	List or add electrical outlet groups
PUT/DELETE	/api/utility-lines/{line}	Update or delete a utility line
PUT/DELETE	/api/generator-lines/{line}	Update or delete a generator line
PUT/DELETE	/api/sockets/{socket}	Update or delete a socket group

Power Analytics

Method	URL	Description
GET	/api/projects/{project}/total-power	Total apparent power (VA) + solar/generator computed values
GET	/api/buildings/{building}/total-power	Same aggregated for a building
GET	/api/floors/{floor}/total-power	Same for a floor
GET	/api/rooms/{room}/total-power	Same for a room
GET	/api/projects/{project}/load-profile	24-hour hourly load array (W per hour, 0–23)
GET	/api/projects/{project}/schedule	Operating schedule summary

Backup, Restore, and Duplicate

Method	URL	Description
GET	/api/projects/{project}/backup	Download full project as JSON file
POST	/api/projects/restore	Restore project from JSON file (with overwrite option)
GET	/api/buildings/{building}/backup	Download building as JSON
POST	/api/projects/{project}/buildings/restore	Restore building into project
GET	/api/floors/{floor}/backup	Download floor as JSON
POST	/api/buildings/{building}/floors/restore	Restore floor into building
GET	/api/rooms/{room}/backup	Download room as JSON
POST	/api/floors/{floor}/rooms/restore	Restore room into floor
POST	/api/projects/{project}/save-backup	Save project snapshot to server database
POST	/api/buildings/{building}/save-backup	Save building snapshot to server
POST	/api/floors/{floor}/save-backup	Save floor snapshot to server
POST	/api/rooms/{room}/save-backup	Save room snapshot to server

GET	/api/projects/{project}/server-backups	List all server-stored backups for project
GET	/api/server-backups/{backup}/data	Retrieve a specific server backup's data
DELETE	/api/server-backups/{backup}	Delete a server backup
POST	/api/projects/{project}/buildings/{building}/duplicate	Clone a building (with all floors, rooms, components)
POST	/api/buildings/{building}/floors/{floor}/duplicate	Clone a floor (any floor in same project) into building
POST	/api/floors/{floor}/rooms/{room}/duplicate	Clone a room (any room in same project) into floor

Project Members

Method	URL	Description
GET	/api/projects/{project}/members	List members + admin (admin-only)
POST	/api/projects/{project}/members	Add member by email (user must already be registered)
PUT	/api/projects/{project}/members/{member}	Change member role
DELETE	/api/projects/{project}/members/{member}	Remove member from project

Admin Panel

Method	URL	Description
POST	/api/admin/login	Admin login (email + password). Returns Sanctum token.
GET	/api/admin/users	List all users in the system
PUT	/api/admin/users/{user}	Edit any user's details
DELETE	/api/admin/users/{user}	Delete a user account

5. Backend Controllers

All 21 API controllers live in `app/Http/Controllers/Api/`.

Controller	Responsibility
AdminController	Admin authentication (email+password) and full user management CRUD.
BuildingController	CRUD for buildings, includes project ownership check.
BuildingComponentController	CRUD for components attached to a building.
ComponentTypeController	Returns list of all component types (presets + custom). Used to populate component-name dropdowns in the frontend.
FloorController	CRUD for floors, returns floor with building + project metadata in index response.
FloorComponentController	CRUD for components attached to a floor.

GeneratorLineController	Polymorphic CRUD for generator backup lines. Accepts parent type + ID, operates on the correct morph relationship.
LoadProfileController	Builds 24-hour hourly load arrays. Sums component power × quantity × schedule overlap. Returns per-hour demand for charting.
NavigationController	Resolves entity names to IDs for breadcrumb navigation.
ProjectBackupController	Full backup/restore/duplicate system. Exports entities to nested JSON; imports JSON back; duplicates entities across the hierarchy.
ProjectComponentController	CRUD for components attached directly to a project.
ProjectController	Main project CRUD. Also provides project-wide all-floors and all-rooms endpoints. Calculates total_power for dashboard listing.
ProjectMemberController	Manages project membership (add by email, change role, remove). Only project admin (owner) can call these.
RoomComponentController	CRUD for components in a room.
RoomController	CRUD for rooms. Index returns rooms with floor + building + project metadata.
ScheduleController	Returns operating schedule summary (work days, hours, seasonal info).
ServerBackupController	Lists, retrieves, and deletes server-stored backup snapshots.
SocketController	Polymorphic CRUD for electrical outlet groups. Uses SocketDemandService for diversity calculations.

TotalPowerController	Aggregates all power loads (components at all levels + sockets) plus solar/generator computed values. Returns max_va, total_va, solar_computed, generator_computed.
UserController	Returns authenticated user info. Handles logout (token revocation).
UtilityLineController	Polymorphic CRUD for utility grid connection lines.

Auth Controller: `Auth/GoogleController` — handles the Google OAuth redirect and callback, creates/updates the user record, and issues a Sanctum bearer token redirected to the frontend.

6. Business Logic Services

BackupExporter.php

Converts any entity into a fully nested JSON snapshot suitable for backup, restore, or duplication. Exports name, area, all components (with every field), utility lines, generator lines, sockets, and recursively exports all children (buildings → floors → rooms).

SourceDispatchService.php

Implements a priority-based dispatch algorithm across 24 hours:

1. **Solar first** — free generation, use as much as available
2. **Utility grid second** — grid electricity at standard tariff
3. **Generator last** — diesel/backup only when solar + grid are insufficient

Returns hourly arrays for each source plus unmet demand and summary statistics used in the PowerBanner component.

SocketDemandService.php

Applies diversified demand factors to electrical outlet groups (based on IEC/NEC diversity):

- Outlets 1–10: **100%** demand factor
- Outlets 11–20: **75%** demand factor
- Outlets 21+: **40%** demand factor

Each outlet has a default of 200 VA. The service also applies a building-wide coincidence factor when pooling across floors.

SolarIrradianceService.php

Estimates hourly solar generation without external API calls:

1. Computes solar declination using Spencer's equation (day of year)
2. Derives sunrise/sunset from hour-angle formula
3. Looks up Peak Sun Hours (PSH) from a built-in table (latitude × month)
4. Distributes PSH as a sinusoidal curve between sunrise and sunset
5. Applies a performance ratio of 0.80 (inverter losses, temperature, soiling)

Returns a 24-element array of watts per hour. Accuracy is $\pm 10\text{--}15\%$ vs measured data.

7. Frontend Pages

Page File	Route / Access	Description
LoginPage.jsx	/login	Landing page with "Sign in with Google" button. Redirects to /dashboard if already authenticated.
AuthCallbackPage.jsx	/auth/callback	Reads the Sanctum token from the URL query string after Google OAuth, stores it in localStorage, navigates to dashboard.
AdminLoginPage.jsx	/admin/login	Email + password login form for the platform administrator.
AdminDashboardPage.jsx	/admin	Table of all registered users with edit (name, email, admin flag) and delete actions.
DashboardPage.jsx	/dashboard	Main user home. Lists all projects (owned + shared) with total power. Create, edit, delete, backup/restore projects. Shows role badge (Admin / Main / View Only).
ProjectPage.jsx	/project	Project overview. Manage buildings (list, add with suggestion dropdown, duplicate, backup). Project-level components, sockets,

		utility/generator lines. Schedule settings. Members management. Power sources banner.
BuildingPage.jsx	/project/building	Building details. Manage floors (list, add, duplicate from any floor in project, backup). Building-level components and sockets. Schedule settings. Power banner.
FloorPage.jsx	/project/building/floor	Floor details. Manage rooms (grid cards, add, duplicate from any room in project, backup). Floor-level components and sockets. Schedule settings. Power banner.
NewRoomPage.jsx	/project/building/floor/room (create)	Full room editor for creating/editing a room. Manage room components with dropdown selector (component type presets), bulk duplicate, time-interval scheduling per component. Sockets management.
RoomPage.jsx	/project/.../rooms (alt view)	URL-param based room view (older navigation style). Lists rooms in a floor with edit/delete.
RoomDetailPage.jsx	/project/building/floor/room (view)	Read-only room analysis view with power summary.
LoadSchedulePage.jsx	/project/load-profile	Interactive 24-hour load profile chart (Recharts). Schedule configuration (work days, time intervals, seasons). Shows hourly demand breakdown.
BuildingsPage.jsx	/project/buildings	Alternate buildings listing view.
FloorsPage.jsx	/project/building/floors	Alternate floors listing view.

8. Reusable Frontend Components

Component	Description
Navbar.jsx	Top navigation bar. Shows user avatar, name, and a logout button. Persists across all protected pages.
UserCard.jsx	Profile card showing name, email, avatar, join date, account type (Google / admin).
ProjectSidebar.jsx	Collapsible left sidebar that shows the project hierarchy tree (project → buildings → floors → rooms) for quick navigation.
PowerBanner.jsx	Sticky top banner showing live power calculations: total load (VA/kVA), solar generation, generator capacity. Refreshes on data changes via a <code>refreshKey</code> prop.
PowerSourcesBanner.jsx	Expandable section showing utility lines, generator lines, solar settings, and dispatch mode for the current entity.
EntityComponents.jsx	Full component management UI (for projects, buildings, floors). Add via dropdown (with auto-fill from component type defaults), edit, delete, duplicate, set usage schedule per component. Handles JSON parse for time intervals.
EntitySockets.jsx	Add/edit/delete electrical outlet groups for any entity. Displays computed diversified demand.
EntityScheduleModal.jsx	Modal for editing work schedule: select days of week, add/remove time intervals, seasonal intervals. Shows inherited schedule from parent entity.
TimeScheduleModal.jsx	Advanced sub-modal for defining precise time intervals (start/end). Used inside EntityScheduleModal and component editors.
BackupChoiceModal.jsx	Two-button modal asking whether to download backup to computer or save to server.

ServerBackupsList.jsx	Lists all server-stored backups for the current project with entity type, date, and a Restore button.
ProjectMembersModal.jsx	Full members management modal: add member by Gmail address (must be registered), change roles, remove members. Shows project admin separately.
LoadingSpinner.jsx	Animated spinning circle. Used as full-page loading state across all pages.

9. Authentication Flow

1. User visits the app and clicks **"Sign in with Google"**.
2. Browser is redirected to `/auth/google` → Laravel redirects to Google's consent screen.
3. User approves → Google redirects to `/auth/google/callback`.
4. Laravel's `GoogleController` calls `Socialite::driver('google')->user()` to get the Google profile.
5. `User::updateOrCreate(['email' => ...])` — creates the user on first login, or updates their name/avatar on subsequent logins.
6. A Sanctum bearer token is generated: `$user->createToken('google-auth')->plainTextToken`.
7. The user is redirected to `http://localhost:5173/auth/callback?token=XXX`.
8. The `AuthCallbackPage` reads the token from the URL, stores it in `localStorage`, and calls `/api/user` to load the profile.
9. All subsequent API calls include `Authorization: Bearer XXX` via the Axios interceptor.

Important: There is no email/password registration. The only way for a user to get an account is to sign in with Google at least once. This means you can only add project members who have already logged in to the platform.

10. Role-Based Access Control

Role	Who Has It	Permissions
Admin	Project creator (owner)	Full control: create/edit/delete all entities, manage members, backup/restore, configure settings

Main	Members assigned by owner	Create/edit/delete entities, backup/restore. Cannot manage members or delete the project.
Normal	Members assigned by owner	Read-only. Can view all data but cannot modify anything.

In the frontend, a `canEdit` flag (`userRole === 'admin' || userRole === 'main'`) gates all add/edit/delete buttons and forms.

11. Backup & Restore System

The system supports backup at four entity levels (project, building, floor, room), each producing a self-contained JSON file.

JSON Structure (example — Floor backup)

```
{
  "version": "1.0",
  "exported_at": "2026-05-15T10:00:00.000Z",
  "floor": {
    "name": "Ground Floor",
    "area": 500,
    "utility_lines": [ { "name": "Main Grid", "power": 50000, "phases": "3phase" } ],
    "generator_lines": [],
    "sockets": [ { "phase_type": "1phase", "power": 200, "quantity": 20 } ],
    "components": [
      {
        "component_name": "LED Panel",
        "power": 40,
        "phases": "1phase",
        "power_factor": 0.9,
        "quantity": 30,
        "group_name": "Lighting",
        "priority": "normal",
        "needs_socket": false,
        "usage_season": "all",
        "usage_day_type": "all",
        "usage_time_intervals": [ { "start": "08:00", "end": "22:00" } ]
      }
    ],
    "rooms": [
      {
        "name": "Office 101",
        "area": 45,
        "components": [ ... ],
        ...
      }
    ]
  }
}
```

```
}  
}
```

Restore Behaviour

- If an entity with the same name already exists, the API returns a `409 Conflict` with a message asking to overwrite.
- If the user confirms, the existing entity is deleted and re-created from the backup.
- All child entities (floors, rooms, components) are restored recursively.

Duplicate Feature

Duplicate creates a copy of any entity (building, floor, or room) within the same project, appending "(Copy)" to the name. Since the project-wide suggestion feature was added, a floor from any building can be duplicated into any other building in the same project.

12. Suggestion Dropdown (Copy From Existing)

When creating a new building, floor, or room, the name field shows a suggestion dropdown populated with existing entities from the *entire project* (not just the current parent).

Each suggestion has two buttons:

- **Empty** — Pre-fills the name and area fields only. Creates a new empty entity.
- **Copy all** — Duplicates the selected entity in full (all components, sub-rooms, etc.) into the current parent. Uses the duplicate API endpoint.

Project-wide endpoints used: `GET /api/projects/{id}/all-floors` and `GET /api/projects/{id}/all-rooms`.

13. Power Calculation Logic

Apparent Power (VA)

For each component: $VA = power \times power_factor \times quantity$

The `TotalPowerController` sums this across all four component tables (project, building, floor, room levels) plus socket demand. Two totals are returned:

- `max_va` — all components regardless of schedule (peak design load)
- `total_va` — components filtered to the current operating schedule (optimised load)

Socket Demand

Outlets are aggregated using diversity factors:

- 1–10 outlets: 100% (e.g., $10 \times 200 \text{ VA} = 2,000 \text{ VA}$)
- 11–20 outlets: 75% additional
- 21+ outlets: 40% additional

Solar Generation

Computed from panel capacity (kW), project location (lat/lng), and month of year. Uses a lookup table of Peak Sun Hours (PSH) per latitude band and distributes output sinusoidally between sunrise and sunset.

Power Dispatch Priority

1. Consume solar first (zero marginal cost)
2. Draw remaining load from utility grid (if solar insufficient)
3. Start generator only when both solar and grid are insufficient

14. Frontend Navigation Architecture

The app uses a **context-based navigation** pattern rather than deep URL parameters for most pages. A lightweight module `navContext.js` stores the current `projectId`, `buildingId`, `floorId`, `roomId` in memory. Pages read these with `getNav()` and write with `setNav()`.

React Router handles top-level routes (`/dashboard`, `/project`, `/project/building`, `/project/building/floor`, etc.). The older URL-param style (used in `RoomPage`) coexists but is being phased out.

Because nav state is in-memory, refreshing a page at `/project/building/floor` loses the building context. This is a known limitation of the current architecture. A future improvement would be to persist IDs in URL params or `sessionStorage`.

15. Local Setup & Deployment

Running Locally

```
# Backend
cd backend
php artisan serve          # runs on http://localhost:8000

# Frontend
cd frontend
npm run dev                 # runs on http://localhost:5173
```

Environment Variables (backend/.env — key values)

```
APP_URL=http://localhost:8000
FRONTEND_URL=http://localhost:5173
DB_CONNECTION=sqlite

GOOGLE_CLIENT_ID=<your-google-client-id>
GOOGLE_CLIENT_SECRET=<your-google-client-secret>
GOOGLE_REDIRECT_URI=http://localhost:8000/auth/google/callback
```

SSL Note (Windows Localhost)

Guzzle (used by Laravel Socialite) fails SSL verification on Windows localhost.

`AppServiceProvider.php` disables SSL verification in the local environment only:

```
if (app()->environment('local')) {
    Socialite::extend('google', function ($app) {
        $config = $app['config']['services.google'];
        return Socialite::buildProvider(GoogleProvider::class, $config)
            ->setHttpClient(new \GuzzleHttp\Client(['verify' => false]));
    });
}
```

Deploying for Other Users

Currently the app runs only on localhost. For other users to access it, the backend must be deployed to a public server (Railway, Render, DigitalOcean, etc.) and the Google OAuth callback URL must be updated in both the Google Cloud Console and `.env`. The SQLite database should be replaced with MySQL or PostgreSQL for production use.

16. Known Limitations & Future Improvements

Area	Current State	Suggested Improvement
Database	SQLite (single file, no concurrency)	Migrate to PostgreSQL or MySQL for production
Auth	Google OAuth only	Add email/password registration so users without Google accounts can sign up

Navigation	In-memory nav state lost on refresh	Persist IDs in URL params or sessionStorage
Deployment	Localhost only	Deploy backend to a cloud server; update Google OAuth redirect
Member Invites	Invitee must sign up before being added	Build an email invitation system
Solar Model	Static PSH lookup table ($\pm 10\text{--}15\%$ accuracy)	Integrate NASA POWER or PVGIS API for real irradiance data
Load Profile	Simplified schedule overlap calculation	Add Monte Carlo simulation for stochastic load modelling
Export	JSON only	Add PDF report generation and Excel export
Real-time	No real-time updates between users	Add WebSockets (Laravel Broadcasting + Echo) for collaborative editing